

Studieområdeprojekt

Lineær Algebra I 3D-Grafik

■■■■■■■■■■ 3.t
NEXT - Københavns Mediegymsnasium

Vejledere:

Matematik A - ■■■■■■ ■■■■■■ ■■■■■■

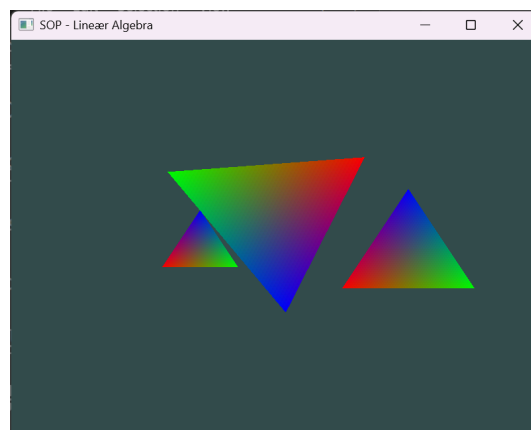
Programmering B - ■■■■■■ ■■■■■■

18.

december 2025

Resumé

Projektet kombinerer en kort historisk redegørelse med en grundig teoretisk gennemgang og en praktisk implementering, inklusive kode eksempler og visualiseringer af, hvordan lineær algebra anvendes til at skabe og manipulere 3D-grafik, og hvilken betydning den har for moderne computerspil. Gennem en kombination af matematisk teori og en praktisk eksempelimplementering i C++ med OpenGL demonstreres, hvordan 3D-punkter transformeres gennem en grafisk pipeline til 2D-pixels. Opgaven gennemgår både abstrakte begreber såsom vektorrum, lineære transformationer, matricer, homogene koordinater og konkrete matrixkonstruktioner, samt praktisk information om grafikort og dets interfaces, og afsluttes ved en diskussion af begrænsninger og fremtiden inden for 3D-grafik.



Indhold

1 Introduktion & Motivation	2
2 3D-Grafik i Computerspil	3
3 Den Teoretiske Del	4
3.1 Lineær Algebra	4
3.2 Vektorrummet	4
3.3 Koordinatsystemet	4
3.4 Lineær Transformation	5
3.5 Matrix	5
3.6 Rumlig Transformationer	8
3.7 Den Fjerde Dimension	8
3.8 Kort om koordinatsystemer	9
3.9 Projektionsmatrixen	9
3.10 Delkonklusion	10
4 Den Praktiske Del	10
4.1 Grafikkortet	10
4.2 Grafikpipelinen	11
4.3 Grafik Interface Krigen	11
4.4 Implementeringsmetode	12
4.5 Kodegennemgang	12
4.5.1 Vertex Shader	13
4.5.2 Applikation	13
4.6 Delkonklusion	15
5 Diskussion	15
5.1 Styrker	15
5.2 Begrænsninger	16
5.3 Pathtracing	16
5.4 Perspektivering	16
5.5 Fremtiden	17
6 Konklusion	17
Bibliografi	18
7 Bilag	19

1 Introduktion & Motivation

Mange spillere bevæger sig igennem komplekse 3D-verdener i videospil uden nogensinde at overveje, hvilke matematiske og teknologiske principper der muliggør sådanne oplevelser. Alle objekters placering består af et komplekst system af tandhjul, der sørger for at en 3D-position bliver til en 2D-pixel på ens skærm. Lineær Algebra er det centrale værktøj der muliggør dette, så objekter kan flyttes, roteres og projiceres. For selvom mange kender til det tryndy *raytracing* i en tid, hvor det er blevet et krav for AAA-titler¹⁵ at komme så tæt på fotorealisme som muligt, er det nødvendigt at kigge tilbage på, hvad der bragte os ind i den tredje dimension til at starte med.

Denne opgave vil tage hånd om både den teoretiske og praktiske del af 3D-grafik med Lineær Algebra i en process kaldet *Rasterisering*. Først vil der være en kort historisk redegørelse af 3D-grafik igennem tiden, hvorefter den matematisk teoretisk del gennemgås, med fokus på matrixmultiplikation, rumlig transformering, projektioner, samt hvordan det relaterer til 3D-grafik. Derefter vil en eksempel implementering i C++ med OpenGL¹⁶ programmeres for at illustrere den praktiske anvendelse af teorien. Til slut vil metodens begrænsninger og lineær algebras generelle anvendelse diskuteres, med en perspektivering af fremtiden for 3D-grafik.

¹⁵Udtalt \triple-a\, er videospil med et stort budget, ofte produceret af industriens førende firmaer med fokus på banebrydende grafik og stor produktionsværdi

¹⁶Et modul til programmering af 2D- og 3D-grafik som virker på tværs af platforme og grafik kort, understøttet af alle grafik kort produceret af AMD, NVIDIA og INTEL

2 3D-Grafik i Computerspil¹⁷

Alene i 2025 er der blevet udgivet 19.000 computerspil på den populære internetbutik *Steam* i følge [SteamDB](#). Derfor må der kigges på nogle helt særlige spil igennem tiden, hvis der skal siges noget om 3D-grafikkens udvikling. I den følgende tidslinje, er en samling af de FPS-spil¹⁸, som alle har revolutioneret 3D-grafik og løftet standarden for spilindustrien på hver sin måde.

1990

Wolfenstein

3D

05-05-1992

Doom

10-12-1993

Quake

31-05-1996

Half-Life

19-11-1998

2000



Figur 2.1: Wolfenstein 3D og Doom af Id Software

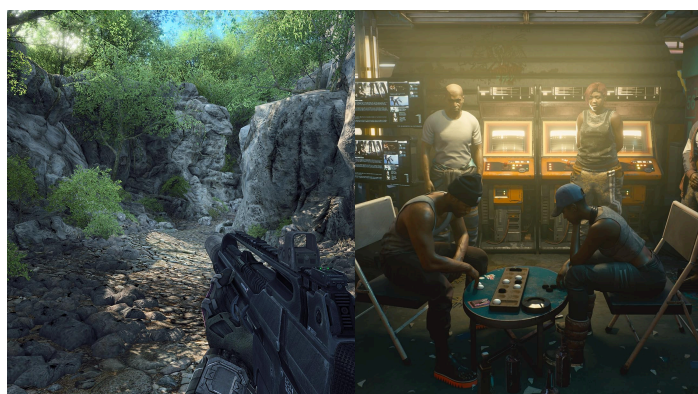
Wolfenstein, sammen med Doom, blev starten på gaming med PC'en¹⁹ og satte for alvor gang i FPS-genren. Disse spil brugte forenkledede metoder, når de tegnede 3D-grafik, og er derfor pseudo-3D. Vægge blev tegnet i vertikale linjer baseret på hvor langt væk de var fra spilleren og monstre var 2D-flader. Disse metoder var hurtige, og nødvendige, da computere var langsomme og grafik kort ikke var udbredte. Metoden har dog flere begrænsninger, så som låst horisontalt perspektiv, alt i et område har den samme mængde belysning, og alle vægge skal være rette linjer.²⁰



Figur 2.2: Quake af Id Software og Half-Life af Valve

Dette ændrede Quake på: der revolutionerede spilindustrien og bragte verdenen fuldkommen 3D og reel belysning! Dette blev muligt af bedre PC'er, men også mulighed for at bruge grafik kort. Nu består verdenen af polygoner der projiceres og tegnes på skærmen gentagende gange i sekundet for at simulere bevægelse. Det er denne proces, rasterisering, som vil blive undersøgt nærmere. Quakes spilmotor blev utrolig populær og eftertragtet af i spilindustrien, og blev derefter bl.a. brugt af Valves til deres populære Half-Life-serie.

2010



Figur 2.3: Crysis af Crytek og Cyberpunk af CD Projekt RED

Efter Quake, bliver størstedelen af 3D-grafik beregnet på grafik kort. Dette giver nye muligheder som spiludviklere hurtigt gør brug af. Standarden for grafik i videospil udvikler sig hurtigt. 10 år efter Quake, i Crysis, er der bløde skygger, global illumination²¹, level of detail, refleksioner og ambient occlusion²². I nyere tid, med spil som *Cyberpunk 2077*, bliver disse metoder kombineret med *pathtracing*, som prøver at simulere lys baglæns for at finde hvilket lys der ses af kameraet.

¹⁷(Lowood, 2014) & (Koirikivi, 2015)

¹⁸**Første Person Skydespil**, hvor man deler øjne med hovedpersonen og bruger skydevåben

¹⁹**Personlig Computer**, en computer designet til personlig brug, har ofte en skærm, tastatur og mus tilsluttet

²⁰(borogk, 2021)

²¹**Global Belysning**, indirekte belysning fra andre objekter medtages (Gregory, 2018, 11.3.3)

²²**Omgivet Tillukning**, en metode der tilføjer skygger i hjørner og tillukkede områder af objekter

3 Den Teoretiske Del

I denne del, skal der beskæftiges med **Lineær Algebra**. Lineær Algebra forbindes ofte med løsning af lineære ligningsystemer, men da der i denne opgave anvendes rasterisering udlukkende i forbindelse med rumlig transformation, vil der kun blive redegjort for vektorrum, lineær transformationer, koordinatsystemer og matricer.

3.1 Lineær Algebra²³

Der er mange matematiske begreber, der får nye navne og definitioner i lineær algebra. Det er derfor vigtigt at disse bliver afklaret. Lineær algebra bliver formuleret over et underliggende *legeme*, som er en mængde, hvor addition og multiplikation er defineret²⁴. Af de mange mulige mængder herunder de reelle tal $\mathbb{R}$ og de komplekse tal $\mathbb{C}$, vil der i denne opgave kun arbejdes med de reelle tal $\mathbb{R}$, da dette er tilstrækkeligt for den 3D-grafik der beskæftiges med. Da betegnelsen *legeme* er relativt abstrakt, vil der i stedet bruges betegnelsen *skalar* eller bare *tal* i denne opgave.

3.2 Vektorrummet²⁵

Vektorer får ligeledes et nyt perspektiv i lineær algebra, hvor der arbejdes med *vektorrum*. Et vektorrum er defineret som en mængde af vektorer, der kan lægges sammen og multipliceres med en *skalar*²⁶. For eksempel, kan det tredimensionelle rumlige vektorrum, som der i denne opgave håndteres mest, beskrives som $\mathbb{R}^3$, hvor hvert element i rummet består af tre reelle tal, et for hver dimension. Disse egenskaber svarer til det som er lært i gymnasiet, og vil derfor også være sådan det behandles i resten af opgaven.

Definition 3.2.1: En vektor $\vec{a}$ af n 'te-dimension er defineret, som en søjle af n reelle tal $\mathbb{R}$, hvorved hvert element får tilnavnet $\vec{a}_1, \vec{a}_2, \dots, \vec{a}_n$

Eksempel: En tredimensionel vektor

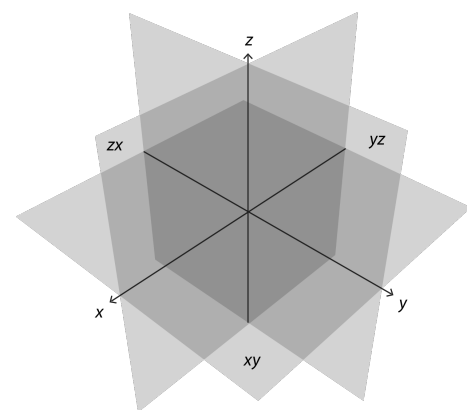
$$\vec{a} = \begin{pmatrix} \vec{a}_1 \\ \vec{a}_2 \\ \vec{a}_3 \end{pmatrix}$$

3.3 Koordinatsystemet²⁷

Vektorrum adskiller den matematiske struktur fra valget af koordinatsystem, og der skal derfor vælges et koordinatsystem, før vektorer kan beregnes og repræsenteres.

I gymnasiet arbejdes der næsten udelukkende med det kartesiske koordinatsystem, hvor akserne står vinkelret på hinanden. Der findes dog andre, bl.a. polære eller sfæriske koordinatsystem.

For et tre dimensionelt kartesisk koordinatsystem, skelnes der desuden mellem venstre- og højrehåndet, baseret på hvordan akserne arrangeres i forhold til hinanden. Hvilken mulighed der vælges har betydning for opfattelsen og visualiseringen af koordinatsystemet og ændrer ikke vektorrummet, det kan dog have betydning for visse retningsbestemte operationer²⁸.



Figur 3.2: De tre planer

²³(Jesper Funch Thomsen, 2020, Kapitel 1 & Bilag A)

²⁴Flere love og definitioner er pålagt (Jesper Funch Thomsen, 2020, Definition A.1)

²⁵(Jesper Funch Thomsen, 2020, Kapitel 5)

²⁶Flere love og definitioner følger (Jesper Funch Thomsen, 2020, Definition 5.1)

²⁷(Gregory, 2018, s. 361; Jensen et al., 2019, Kapitel 1.1) & (Lengyel, 2016, Kapitel 2)

²⁸Eksempelvis krydsproduktet, som dog ikke anvendes i denne opgave.

Dette illustreres ved brug af *håndreglen* og **Figur 3.2**. Ved at forme en knytnæve med venstre hånd og lade tommelfingeren pege i z-aksens positive retning, vil fingrespidsernes retning angive x-aksens positive retning, mens det første led fra knoerne peger i y-aksens positive retning. Koordinatsystemet i figuren er derfor venstre håndet. Ligeledes kan denne test foretages med højrehånd. I denne opgave vil der både blive brugt en venstre og højrehåndet opfattelse af det kartesiske koordinatsystem, da dette er standarden i OpenGL, selvom det plejer at være udelukkende venstre håndet.²⁹

3.4 Lineær Transformation³⁰

Når vektorer skaleres, kan transformationen opfattes som en forstørrelse af hele rummet, hvor alle vektorer bliver forstørret eller forkortet. En sådan transformation beskrives i lineær algebra af afbildninger, som virker på vektorer i et vektorrum. Hvis en afbildning opfylder **Definition 3.4.1**, kaldes den for en lineær afbildning/transformation. En lineær afbildning er en udvidelse af den lineære funktion fra gymnasiet til at håndtere vektorrum, hvor den beskriver transformation mellem vektorrum.

Definition 3.4.1: En lineær afbildning f mellem vektorrum bevarer addition og skalarmultiplikation for alle vektorer $\vec{a}$ og $\vec{b}$, samt skalarer α .

$$f(\vec{a} + \vec{b}) = f(\vec{a}) + f(\vec{b})$$

$$f(\alpha \vec{a}) = \alpha f(\vec{a})$$

3.5 Matrix³¹

En lineær transformation kan repræsenteres som en matrix. Derfor bruges matricer ofte til at repræsentere rumlige transformationer, da de er effektive at beregne og gør det muligt at lave en kombination af transformering. Især i 3D-grafik bruges matricer ofte, og udgør derfor fokuset for resten af den teoretiske del.

Definition 3.5.1: En matrix $\mathbf{M}_{n \times m}$, er en tabel af tal med n rækker og m søjler. Når $n = m$ kaldes matrixen for kvadratisk. Hvert tal/element i matrixen får tilnavnet $\mathbf{M}_{ij}$, for i er tallets række og j er dets søjle.

Eksempel:

$$\mathbf{M}_{3 \times 3} = \begin{bmatrix} \mathbf{M}_{11} & \mathbf{M}_{12} & \mathbf{M}_{13} \\ \mathbf{M}_{21} & \mathbf{M}_{22} & \mathbf{M}_{23} \\ \mathbf{M}_{31} & \mathbf{M}_{32} & \mathbf{M}_{33} \end{bmatrix}$$

Definition 3.5.2: En identitets matrix $\mathbf{I}_n$, er en kvadratisk matrix med størrelse n , dvs. n søjler or rækker, hvor alle tal i diagonalen er lig med et:

$$\mathbf{I}_{ij} = \begin{cases} 1 & \text{hvis } i = j \\ 0 & \text{hvis } i \neq j \end{cases}$$

Eksempel:

$$\mathbf{I}_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Definition 3.5.3: En vektor i et n -dimensionelt vektorrum, kan repræsenteres som en søjle matrix $\mathbf{M}_{n \times 1}$, med n rækker og en søjle.

$$\vec{a} = \begin{pmatrix} \vec{a}_1 \\ \vdots \\ \vec{a}_n \end{pmatrix} = \begin{bmatrix} \vec{a}_1 \\ \vdots \\ \vec{a}_3 \end{bmatrix}$$

Derudover er det værd at vide, at når en kvadratisk matrix $\mathbf{M}$ multipliceres med en vektor $\vec{v}$ (repræsenteret som en søjlevektor), vil resultatet altid være en vektor, som har gennemgået den lineære transformation som $\mathbf{M}$ repræsenterer.

²⁹(Gregory, 2018, s. 362)

³⁰(Jesper Funch Thomsen, 2020, Kapitel 6)

³¹(Lengyel, 2016, Kapitel 3)

Matricer kan, ligesom vektorer, lægges sammen og multipliceres med en skalar, så længe at dimensionerne passer.

Derudover har matricer også en særlig form for multiplikation, kaldet matrixmultiplikation, som er givet i

Definition 3.5.4. Matrixmultiplikation er særlig interessant, da den svarer til at kombinere lineære transformationer. Desuden, vil identitetsmatrixen, defineret i **Definition 3.5.2**, fungere som et neutralt element. Det er vigtigt at understrege, at matrixmultiplikation, i modsætning til normal multiplikation, generelt ikke er kommutativt, og rækkefølgen af transformationer har således en effekt for resultatet.

$$\mathbf{M} \times \mathbf{G} \neq \mathbf{G} \times \mathbf{M}$$

Definition 3.5.4: Givet to matricer $\mathbf{M}_{n \times m}$ og $\mathbf{G}_{m \times p}$ er produktet af matrix multiplikation endnu en matrix $\mathbf{F}_{n \times p}$, hvorved hvert tal i produktet er summen af produkterne mellem dets tilsvarende række i $\mathbf{M}$ og søjle i $\mathbf{G}$, og defineres matematisk således ved hjælp af sum symbolet Σ :

$$\mathbf{F}_{ij} = \sum_{k=1}^m \mathbf{M}_{ik} * \mathbf{G}_{kj}$$

Eksempel:

$$\mathbf{M}_{2 \times 2} \times \mathbf{G}_{2 \times 2} = \mathbf{F}_{2 \times 2} = \begin{bmatrix} \mathbf{M}_{11} & \mathbf{M}_{12} \\ \mathbf{M}_{21} & \mathbf{M}_{22} \end{bmatrix} \times \begin{bmatrix} \mathbf{G}_{11} & \mathbf{G}_{12} \\ \mathbf{G}_{21} & \mathbf{G}_{22} \end{bmatrix} = \begin{bmatrix} \mathbf{M}_{11} * \mathbf{G}_{11} + \mathbf{M}_{12} * \mathbf{G}_{21} & \mathbf{M}_{11} * \mathbf{G}_{12} + \mathbf{M}_{12} * \mathbf{G}_{22} \\ \mathbf{M}_{21} * \mathbf{G}_{11} + \mathbf{M}_{22} * \mathbf{G}_{21} & \mathbf{M}_{21} * \mathbf{G}_{12} + \mathbf{M}_{22} * \mathbf{G}_{22} \end{bmatrix}$$

Sætning 3.5.1: Produktet af matrixmultiplikation for en kvadratisk matrix $\mathbf{M}_{n \times n}$ og identitets matrixen af samme størrelse $\mathbf{I}_n$ vil altid være identisk til den originale matrix $\mathbf{M}_{n \times n}$.

$$\mathbf{M}_{n \times n} \times \mathbf{I}_n = \mathbf{I}_n \times \mathbf{M}_{n \times n} = \mathbf{M}_{n \times n}$$

Bevis: Ifølge definitionen af matrixmultiplikation fra **Definition 3.5.4**, kan hvert element i produktet, mellem matrix $\mathbf{M}$ og identitetsmatrix $\mathbf{I}$ skrives som

$$(\mathbf{M} \times \mathbf{I})_{ij} = \sum_{k=1}^n \mathbf{M}_{ik} * \mathbf{I}_{kj}$$

Derefter indsættes definitionen af identitetsmatrixen **Definition 3.5.2** og fås:

$$(\mathbf{M} \times \mathbf{I})_{ij} = \sum_{k=1}^n \mathbf{M}_{ik} * \begin{cases} 1 & \text{hvis } k = j \\ 0 & \text{hvis } k \neq j \end{cases}$$

Da alle led i summen er nul når $k \neq j$, kan de ignoreres dem, og der reduceres derfor, så led $k = j$ står tilbage. Derefter erstattes k med j :

$$(\mathbf{M} \times \mathbf{I})_{ij} = \mathbf{M}_{ij} * 1 = \mathbf{M}_{ij}$$

Da dette gælder for alle elementer, må det betyde at:

$$\mathbf{M} \times \mathbf{I} = \mathbf{M}$$

Tilsvarende, kan det vises at hvilken hvilken side identitetsmatrixen opstår på ikke har en effekt, dvs. at:

$$\mathbf{I} \times \mathbf{M} = \mathbf{M}$$

Sætningen er dermed bevist. ■

Sætning 3.5.2: Lad $\mathbf{M}$ og $\mathbf{G}$ være to kvadratiske matricer af størrelse $n \times n$, som repræsenterer lineære transformationer i et n -dimensionelt vektorrum. For enhver vektor $\vec{v}$ i dette vektorrum, kan den samlede lineære transformation F , som opnås ved først at anvende $\mathbf{G}$ og derefter $\mathbf{M}$, fås ved matrixmultiplikation:

$$\mathbf{F}\vec{v} = \mathbf{M} \times (\mathbf{G} \times \vec{v})$$

Bevis: Ligesom i det tidligere bevis, anvendes definitionen for matrixmultiplikation givet i **Definition 3.5.4**. Da en lineær transformation af en vektor resultere en søjlematrix (se **Definition 3.5.3**), fokuseres der her på det i -te element.

$$(\mathbf{M} \times (\mathbf{G} \times \vec{v}))_i = \sum_{k=1}^n \mathbf{M}_{ik} * (\mathbf{G} \times \vec{v})_k$$

Indsæt matrixmultiplikation definitionen for $\mathbf{G} \times \vec{v}$ og byt rækkefølgen af summationerne, hvilket er muligt fordi sum er kommutativt og associativt:

$$\begin{aligned} (\mathbf{M} \times (\mathbf{G} \times \vec{v}))_i &= \sum_{k=1}^n \mathbf{M}_{ik} * \left(\sum_{j=1}^n \mathbf{G}_{kj} * \vec{v}_j \right) \\ &= \sum_{k=1}^n \sum_{j=1}^n \mathbf{M}_{ik} * \mathbf{G}_{kj} * \vec{v}_j \\ &= \sum_{j=1}^n \sum_{k=1}^n \mathbf{M}_{ik} * \mathbf{G}_{kj} * \vec{v}_j \\ &= \sum_{j=1}^n \left(\sum_{k=1}^n \mathbf{M}_{ik} * \mathbf{G}_{kj} \right) * \vec{v}_j \end{aligned}$$

Det indre sum led, som følger matrixmultiplikation definitionen for $\mathbf{M}$ og $\mathbf{G}$, erstattes:

$$\begin{aligned} (\mathbf{M} \times (\mathbf{G} \times \vec{v}))_i &= \sum_{j=1}^n \vec{v}_j * (\mathbf{M} \times \mathbf{G})_{ij} \\ &= ((\mathbf{M} \times \mathbf{G})\vec{v})_i \end{aligned}$$

Da dette gælder for alle rækker i i søjlematricen, og der fås derved:

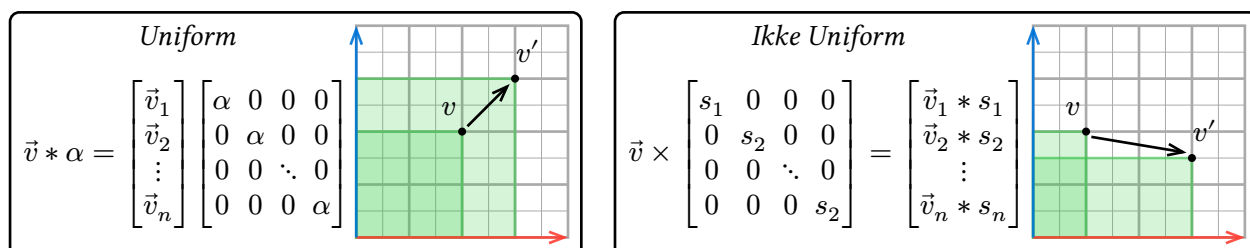
$$\mathbf{M} \times (\mathbf{G} \times \vec{v}) = (\mathbf{M} \times \mathbf{G})\vec{v} = \mathbf{F} \times \vec{v}$$

Sætningen er dermed bevist. ■

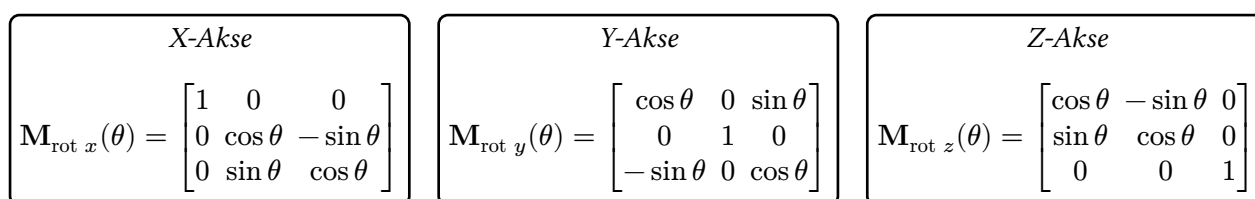
3.6 Rumlig Transformationer³²

I 3D-grafik ønskes der ofte at flytte, skalerer og roterer objekter. Dette gøres selvfølgelig med matricer. Hvordan disse matricer ser ud vil der blive redegjort for her. Alle matricerne er kvadratiske og kommer til at blive brugt på vektorer.

Først og fremmest den mest simple: skaleringsmatricen. Denne kvadratisk matrice kan bruges til at forstørre eller formindske rummet. En skalering, som er lige på alle elementer, kaldes for uniform, hvilket er lig skalarmultiplikation for vektorer. Skaleringen er givet langs diagonalen.



Den anden lineære transformation, som der vil blive gjort brugt, er rotations matricer. Her vil der blive defineret for tredje-dimension, altså rundt om x-, y- og z-aksen. Rotationen er givet af en vinkel θ rundt om aksen. Det er vigtigt at huske, at matrixmultiplikation ikke er kommutativt, og derfor har rækkefølgen for hvordan rotationerne sammensættes betydning.



3.7 Den Fjerde Dimension³³

Selvom meget kan gøres med skalering og rotering, sker det stadig ofte at et ønske om at flytte objekter opstår. Dette er dog ikke lineært, per [Definition 3.4.1](#); da lineær afbildning skal opfylde $f(\vec{0}) = \vec{0}$. I stedet når der flyttes, kaldet translation, opstår ikke linearitet: $f(\vec{0}) = \vec{t} \neq \vec{0}$

Derfor kan denne rumlige transformering ikke repræsenteres med en 3×3 matrix. Men hvad at tilføje en fjerde-dimension, kaldt for homogene koordinater, kan både translation samt andre rumlige transformeringer repræsenteres i en enkelt matrice.

Matrixen tager denne form, hvor $\mathbf{M}$ er en anden rumligt transformation, såsom skalering, og $\vec{v}$ er det der ønskes at flyttes med. Bemærk, at de rumlige transformationer stadig kan kombineres med matrixmultiplikation.

For vektorer, får de en fjerde akse w . Hvert element i vektoren skal divideres med denne værdi, før den kan repræsenteres i 3D. Derudover kan de vælges om en vektor skal behandle translation, ved at sætte w til 0 eller 1. I tilfælde hvor vektorer repræsenterer en retning, vil w blive sat til 0 inden transformation, og omvendt hvis vektoren repræsenterer et punkt, vil w sættes til 1.

Definition 3.7.1: En 4×4 matrix, med en lineær afbildning repræsenteret i matrix M og translation med vektor v har følgende form:

$$\mathbf{F} = \left[\begin{array}{ccc|c} \mathbf{M}_{11} & \mathbf{M}_{12} & \mathbf{M}_{13} & \vec{v}_1 \\ \mathbf{M}_{21} & \mathbf{M}_{22} & \mathbf{M}_{23} & \vec{v}_2 \\ \mathbf{M}_{31} & \mathbf{M}_{32} & \mathbf{M}_{33} & \vec{v}_3 \\ \hline 0 & 0 & 0 & 1 \end{array} \right]$$

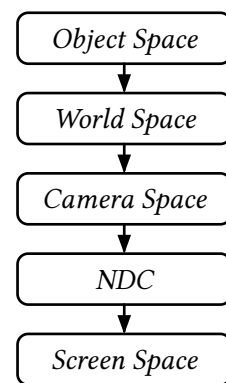
³²(Lengyel, 2012, s. 54 & s. 61)

³³(Lengyel, 2012, s. 65-68)

3.8 Kort om koordinatsystemer³⁴

Punkterne i 3D-grafik gennemgår før en lang række af transformationer de når skærmen. Indtil videre er der blevet opereret i *object space*. Her er punkter defineret i forhold til den geometri de udgør. Punkterne kan blive flyttet rundt med de tidligere beskrevet rumlige transformationer, i det der hedder *modelmatricen*, så de havner i *world space*. Det vil altså sige, at i spil hvor man løber rundt, forbliver karakterens repræsentation stille. Det er i stedet dets *modelmatrice* som flytter den rundt i verdenen.

Tilskuerne skal have et indblik til den 3D-verden, hvilket gøres med et virtuelt kamera. Alt geometrien bliver sat i forhold til det i *camera space* med en *viewmatrix*. Her alle punkterne i forhold til kameraet, så origo ligger ved kameraets position. Hvordan virtuelle kameraer og *viewmatrix* virker vil ikke blive undersøgt i denne opgave.



Figur 3.13: Kæden af transformationer fra 3D-punkt til 2D-skærm

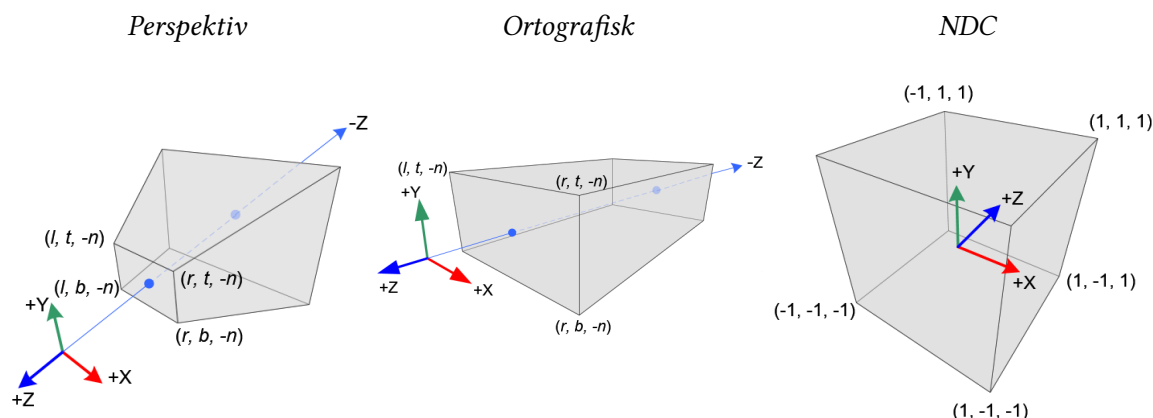
Herfra bliver koordinatsystemet projekteret ind i en boks kaldet *NDC* (*normalized device coordinates*), med en størrelse fra -1 til $+1$ på alle akser. Alle punkter der projekteres uden for boksen kan ikke ses af det virtuelle kamera. I dette koordinatsystem udgør $(0, 0)$ centeret for skærmen, hvilket er det sidste koordinatsystem.

3.9 Projektionsmatrixen³⁵

I 3D-grafik er det ikke tilstrækkeligt blot at flytte, skalerer og roterer punkter. For at kunne vise et tre-dimensionelt punkt på en 2D-skærm, skal der udføres en projektion.

Det er projektionsmatrixens formål at gå fra at 3D-punkt til NDC. Der bruges oftest to af de følgende projektioner: perspektiv eller ortografisk. Forskellen er blot om geometriens afstand til kameraet har betydning for dets størrelse. Projektionsmatrixen nær- og fjerntplan udgør hvor 'fokuset for kameraet', alt inden og efter er uden for rækkevide.

Ved anvendelse af homogene koordinater kan også projektion formuleres som en matrixtransformation. Projektionsmatrixen er derfor ligeledes en 4×4 matrix, som anvendes på samme måde som de øvrige rumlige transformationer. I praksis indgår projektionsmatrixen som det sidste led i kæden, efter objektets position og orientering er fastlagt. Hvordan projektionsmatrixerne er blevet til vil ikke blive undersøgt i denne opgave.



Figur 3.14: Illustration af perspektiv og ortografisk projektioner, samt NDC
Billeder stammer fra følgende [hjemmeside](#)

³⁴(Gregory, 2018, Afsnit 5.3.9 & 11.1.4.1)

³⁵(Gregory, 2018, Afsnit 11.1.4.2)

Definition 3.9.1: Givet nær-planets venstre l , højre r , top t og bund b . Samt afstanden til nær- n og fjernplanet f . Kan en perspektivmatrix $\mathbf{P}$ og en ortografiskmatrix $\mathbf{O}$ opstilles således.

$$\mathbf{P} = \begin{bmatrix} \left(\frac{2n}{r-l}\right) & 0 & 0 & 0 \\ 0 & \left(\frac{2n}{t-b}\right) & 0 & 0 \\ \left(\frac{r+l}{r-l}\right) & \left(\frac{t+b}{t-b}\right) & \left(-\frac{f+n}{f-n}\right) & \left(-\frac{2nf}{f-n}\right) \\ 0 & 0 & -1 & 0 \end{bmatrix} \quad \mathbf{O} = \begin{bmatrix} \left(\frac{2n}{r-l}\right) & 0 & 0 & \left(-\frac{r+l}{r-l}\right) \\ 0 & \left(\frac{2n}{t-b}\right) & 0 & \left(-\frac{t+b}{t-b}\right) \\ 0 & 0 & \left(-\frac{2}{f-n}\right) & \left(-\frac{f+n}{f-n}\right) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3.10 Delkonklusion

I denne teoretiske gennemgang er fundamentet for 3D-grafik og rasterisering med lineær algebra blevet lagt. Der er blevet redegjort for grundprincipperne i lineær algebra, og hvordan nogle objekter differentiere sig fra gymnasie-pensum. Der er særligt blevet lagt væk på matricer, og deres egenskab til repræsenterer lineære transformationer.

4 Den Praktiske Del

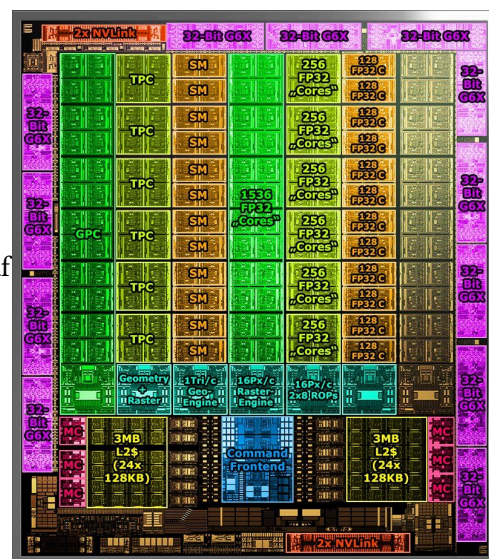
Med denne teori under armen, skal der nu kigges nærmere på hvordan dette anvendes. Dette vil blive gjort i den følgende rækkefølge: *hardware* $\Rightarrow$ *interface* $\Rightarrow$ *eksempel applikation*.

4.1 Grafikkortet³⁶

Overgangen fra pseudo-3D i *Doom* til fuld 3D-grafik i *Quake* blev starten på et teknologisk fremskridt, der gav plads til nye komponenter i PC'en. *Quake* blev netop et af de første store hits som understøttede grafikkort³⁷. Dette markerede starten på grafikkortets centrale rolle i 3D-grafik. Men hvad er et grafikkort egentligt?

Et grafikkort, også kendt som en **GPU** (*graphics processing unit*), er et specialiseret stykke hardware designet til at beregne store mængder af data parallelt. I moderne grafikkort består hver microchip³⁸ af flere tusinde mindre beregningsenheder, kaldet *kerner*. Et eksempel på denne opbygning for et moderne grafikkort ses i **Figur 4.1**.

Alle disse *kerner* kan programmeres på en måde, der minder om CPU'en (*central processing unit*). Dog skal alle kernerne programmeres til det samme. Derfor egner grafikkort sig til ensartet uafhængigt arbejde, som kan distribueres til de mange kerner. En typisk vil disse kerner lave beregninger på decimal tal, repræsenteret af *floating point*-numre digitalt.



Figur 4.1: Overblik af NVIDIA's GA102 (Ampere) Mikrochip³⁹

Inden for 3D-grafik er dette især relevant for de tidligere nævnte *lineære transformationer*, da denne operation er enormt effektiv at beregne i parallelt. Derfor kan grafikkort indeholde specielle enheder, kaldt *tensors*, der er optimeret til udelukkende matrixmultiplikation.

³⁶(Gregory, 2018, Kapitel 4.2 & 11.2)

³⁷[Quake \(video game\) - Wikipedia](#)

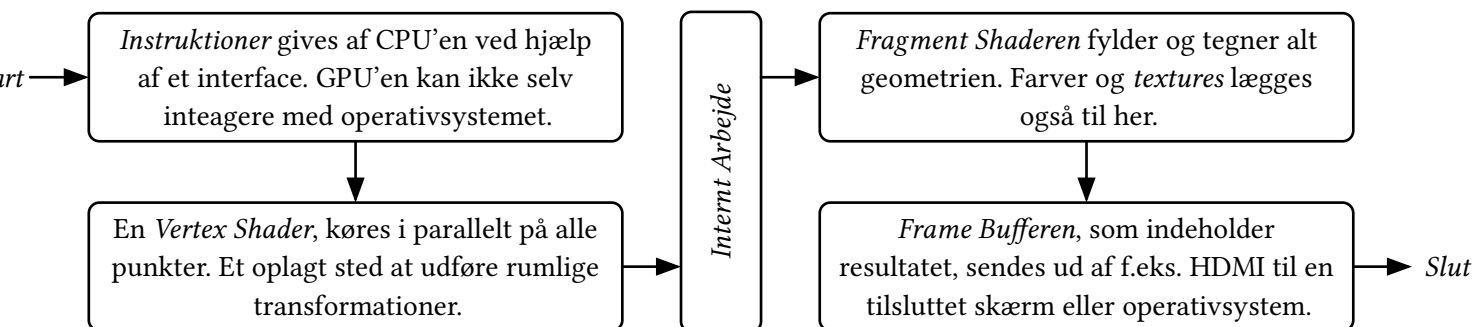
³⁸**Mikrochip**, er et integreret kredsløb skabt til f.eks. at beregne eller andet digitalt arbejde

³⁹[IR Photographer Shares Die Shots of Nvidia 3000 Series GA102 Silicon - Tom's Hardware](#)

Flops (floating point beregninger pr. sekund), kan bruges til at måle kraften af et grafikkort. Grafikkort som kørte *Quake* havde MFLOPS (10^6 , million), mens moderne grafikkorts beregningskraft måles i TFLOPS (10^{12} , billion).

4.2 Grafikpipelinen⁴⁰

Processen fra 3D-geometri bestående af objekter til et færdigt billede kaldes for *pipelinen* og udføres på grafikkortet. De programmerbare dele af pipelinen skrives i *shaders*, som udføres i parallel. Den følgende figur illustrere hvilke dele af pipelinen der vil blive kigget nærmere på i denne opgave.



Figur 4.2: Illustration af en mulig grafik pipeline

4.3 Grafik Interface Krigen⁴¹

Som antydnet i figuren ovenover, starter programmeringen af grafikkort med CPU'en. Kommunikationen mellem CPU'en tager følgende form:



GPU'en og Driveren håndteres af af grafikkort fabrikanten. Centralt for dette er dog at driverne ofte er udviklet til et særligt operativsystem. Lige netop operativsystemet, og dets forskelle viser sig at blive besværligt for grafikkort. For i modsætning til CPU'er, hvis instruktioner ofte er standardiseret, f.eks. *x86*, *ARM* og *amd64*, så er der et vel af forskellige interfaces til programmeringen af GPU'er. Der findes mange åbne og lukkede standarder, så her er de mest mærkbare:

OpenGL blev standardiseret i 1992 og derefter senere udviklet af Khronos Group. Dette var den første reele standardt til graffikort, eller 3D-acceleratorer som de blev kaldt den gang. OpenGL mager dog nogle moderne ønsker, såsom raytracing, men er til gengæld understøttet af stort set alle grafikkort. Dog bliver denne standardt ikke udviklet længere og er blevet afløst af *Vulkan*

Vulkan er også et interface standardiseret af Khronos Group. Dette interface giver langt mere eksplicit kontrol og er derfor en smule indviklet, mod at være mere kraftfuld, og gør det muligt for udviklere at optimere bedre. Vulkan har har flere moderne funktioner en OpenGL, fx. raytracing.

DirectX er, i modsætning til de tidligere nævnte, fuldkommen eksklusivt interface og er udviklet af *Microsoft*. Det er derfor udlukkende understøttet i *Windows* og til *Xbox'en*

Metal er *Apples* eksklusive interface, som skal erstatte OpenGL på deres operativsystemer.

⁴⁰(Gregory, 2018, Afsnit 11.2.4)

⁴¹(eisfrosch, 2025)

⁴²**Applikation Programmerings Interface** eller **programmeringsgrænseflade**, er en standardiseret mellemmand, som kan facilitere kommunikation mellem to programmer.

Det er *OpenGL* der vil blive brugt til eksempel programmet, da det understøttes af flest mulige platforme og har mest udbredt understøttelse. *Vulkan* kunne også bruges, men dets explicite natur gør at eksempel programmet kunne fylde op til 9 gange så mange linjer⁴³.

4.4 Implementeringsmetode⁴⁴

For at interagere med OpenGL skal der først og fremmest skrives et program til CPU'en. Dette program skal håndtere flere ting: skabe et vindue til brugeren, opsætning af OpenGL og give kommandoer til GPU'en.

Der er mange forskellige programmeringssprog som kan bruges. De differentieres ofte baseret på om de er *kompileret* eller *fortolket*. Et kompileret programmeringssprog kræver at udvikleren bruger en *oversætter* til at gå fra den læsbare tekst til binært maskinsprog. I kompilerede sprog, er det ofte et krav, at programmøren selv holder styr på computerens hukommelse. Dette er i modsætning til fortolket sprog, hvor teksten bliver behandlet af et andet program når koden skal køres. Fortolkede sprog har ofte en *garbage collector*, som håndterer programmets hukommelse. Kompilerede sprog er ofte hurtigere end fortolkede sprog, men kræver at koden bliver oversat til de platforme hvor det skal køres.

Derfor er det oplagt at bruge det kompilerede programmeringssprog *C++*, som originalt blev udviklet af danskeren Bjarne Stroustrup. Dette sprog er baseret på 'moderen' af alle programmeringssprog *C*, og kan derfor bruges som en udvidelse af det. *C++* er fordelagtigt til computergrafik da det giver programmøren tæt kontrol og er designet til at lave høj hastigheds applikationer. Desuden har dette været det mest brugte programmeringssprog i spilindustrien.

I denne opgave anvendes *C++* udelukkende som et redskab til at interagere med OpenGL. Derfor vil der ikke blive redegjort for sprogets syntaks og opførsel. Dertil gøres der opmærksom på at der bliver gjort brug af *glfw* og *glad* til at skabe et vindue og håndtere opsætningen af OpenGL.

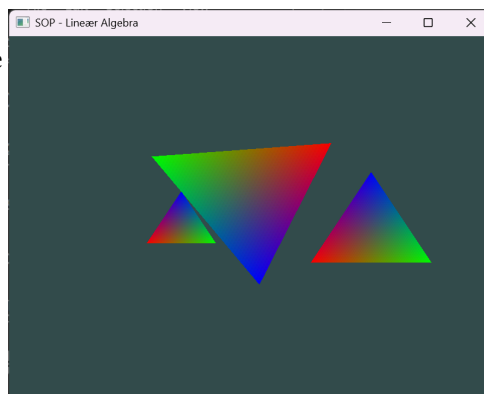
Derudover bruges programmeringssproget *GLSL* (*OpenGL shader language*), til at skrive shadersne fra *pipelinen*.

Den grundlæggende OpenGL-opsætning er baseret på en gennemgang beskrevet i (De Vries, 2020), som primært anvendes til initialisering af interfacet. Kodegennemgangen vil derfor fokusere på hvordan de matematiske principper bindes sammen med OpenGL.

4.5 Kodegennemgang

Eksempel applikationen, kommer til at vise de rumlige transformationer, som der blev redegjort for i den teoretiske del. Altså de følgende matricer: skalering, rotation og translation.

For at illustrere disse, vil der blive tegnet 3 trekanter ved siden af hinanden. Den første, til venstre, vil blive skubbet frem og tilbage på z-aksen mens den gør brug af en perspektivprojektering. Den midterste, vil blive skaleret og roteret rundt. Og den til højre skubbes også på z-aksen, ligesom den til venstre, men bruger derimod en ortografisk projektering. Dette ses i Figur 4.3. Bemærk dog, at det forventede resultat for den højre trekant, er at der sker intet visuelt med tiden.



Figur 4.3: Eksempel applikationen

⁴³(eisfrosch, 2025, 7:46-7:51)

⁴⁴(Gregory, 2018, Afsnit 3.1.2)

Der er mange måder, hvorpå matricer kan implementeres på grafikort med OpenGL. I denne opgave vil der tages udgangspunkt i CPU'en med modulet *glm*.

4.5.1 Vertex Shader

Der vil blive startet med at gennemgå, hvordan *glm* kan bruges. Først og fremmest skal der skrives en vertex shader i *GLSL*. Vertex shaderen bliver kørt på alle de punkter som tegnes, og er derfor stedet hvor der laves rumlige transformeringer. Sådan ser koden ud:

```
layout (location = 0) in vec3 inPos;
layout (location = 1) in vec3 inColor;

uniform mat4 projection;
uniform mat4 model;

out vec3 outColor;

void main()
{
    gl_Position = projection
                  * model
                  * vec4(inPos, 1.0f);
    outColor = inColor;
}
```

Der defineres først hvilket data som shaderen forventer. Her er de to tre-dimensionelle vektorer, hvor den ene repræsenterer punktet der skal transformeres og den anden en RGB-værdi til farven af punktet. Derefter specificeres to global matricer, som skal bruges til transformationen, *projection* og *model* som nævnt i [Afsnit 3.8](#)

I *main()*, skrives instruktionerne der skal udføres på grafikortet. Her kan det ses at variabelen *gl_Position*, sættes til at være resultatet af matrixmultiplikation for først modelmatrixen og punktet, og derefter det sidste led, projectionsmatrixen. *gl_Position* er derved det originale punkt i *NDC*. Som tidligere nævnt er OpenGL både venstre- og højrehåndet, hvor *NDC* er venstre og *world space* er højre.

Da der arbejdes med homogene koordinater sættes det fjerde komponent til at være 1.0, ligesom beskrevet i [Afsnit 3.7](#). Læg mærke til rækkefølgen for multiplikationen. Byttes der rundt på den får man ikke det korrekte resultat.

4.5.2 Applikation

Inden der kan gives kommandoer til OpenGL, skal der defineres nogle konstanter. Bemærk dog at en dynamisk liste i C++ hedder en `std::vector`, og ikke repræsenterer et punkt.

```
// Forskydningen for de tre trekanter
std::vector<glm::vec3> trianglePositions = {
    glm::vec3(-0.5f, 0.0f, 0.0f),
    glm::vec3(0.0f, 0.0f, 0.0f),
    glm::vec3(0.5f, 0.0f, 0.0f)
};

// Perspektiv- og ortografiskmatrix
// Begge funktioner specificerer en et nær-plan med størrelsen -1 til 1
// og at nær- og fjernplanet er henholdsvis 1 og 10 væk fra origo.
glm::mat4 perspective = glm::frustum(-1.0f, 1.0f, -1.0f, 1.0f, 1.0f, 10.0f);
glm::mat4 orthographic = glm::ortho(-1.0f, 1.0f, -1.0f, 1.0f, 1.0f, 10.0f);

// Tiden siden starten af programmet blev kørt.
// Denne værdi vil blive opdateret løbende
float time = (float)glfwGetTime();
```

I en *while*-lykke, som kører hver frame, kan der specificeres hvordan indholdet af vinduet skal tegnes. Til de tre trekanter, er der tidligere blevet specificeret deres forskydning i en liste af vektorer. Denne liste itereres over for at tegne hver trekant og tager følgende form:

```
for (size_t i = 0; i < trianglePositions.size(); ++i) {
    // Indhold
}
```

Selve indholdet er brudt ned i dele for at gøre gennemgangen lettere. Der skal først og fremmest specificeres hvilken projektion der skal bruges til trekanten.

```
// Der startes med at definere en tom 4x4 matrix
glm::mat4 projection;
switch (i) {
    case 0:
        projection = perspective;
        break;
    case 1:
        // Som tidligere bevist i Sætning 3.5.1 har identitets matrixen ingen effekt
        i sig selv
        projection = glm::identity<glm::mat4>();
        break;
    case 2:
        projection = orthographic;
        break;
}
glUniformMatrix4fv(projectionLoc, 1, GL_FALSE, glm::value_ptr(projection));
```

Her bliver der gjort brug af en *switch*-erklæring. Som input tager den trekantens nummer *i*, hvor 0 er trekanten til venstre og 2 er den til højre. Når koden gøres tjekkes inputet og den rette case vil blive fundet og kørt. Inde i hver case, sættes værdien for projekteringen. Tilslut bliver matricen sendt til vertex shaderen. Derefter mangler blot modelmatricen:

```
// Der startes med at definere en 4x4 identitetsmatrix
glm::mat4 transform = glm::identity<glm::mat4>();

/* Det er vigtigt at betragte rækkefølgen af de følgende matrixmultiplikationer.
   Da ønsket er først at skalere trekanten, derefter rotere rundt om sig selv,
   og til slut forskydes, bliver rækkefølgen af matrixmultiplikationerne
   behandlet omvendt, pr. Sætning 3.5.2. Translationsmatrixen multipliceres først
   i kombinationen, men udføres til sidst i helheden af transformationen.
   Hver transformation gør brug af det sidste resultat og overskriver det. */

// Hvis det er den venstre eller højre trekant
if (i == 0 || i == 2) {
    // Den prædefinerede forskydning lægges sammen med en z-afstand givet tiden.
    float zdistance = sin(time) * 2.0f - 3.0f;
    transform = glm::translate(
        transform,
        trianglePositions[i] + glm::vec3(0.0f, 0.0f, zdistance));
} else { // Hvis det hverken er venstre eller højre, så må det være midten

    // Translation med den prædefinerede forskydning for netop denne trekant
    transform = glm::translate(transform, trianglePositions[i]);

    // Rotation rundt om Z-aksen. Vinklen er baseret på tiden, hvor hver trekant
    // er forskudt med 1 sekund, så det tager 6 sekunder for en fuld omdrejnings.
    float angle = (time + (float)i) * glm::radians(60.0f);
    transform = glm::rotate(transform, angle, glm::vec3(0.0f, 0.0f, 1.0f));

    // Skaleringen er også givet over tid.
```



```
// Her specificeres det at skaleringen er på alle akser.
float scale = sin(time + i) * 0.5f + 1.0f;
transform = glm::scale(transform, glm::vec3(scale, scale, scale));
}

// Værdien af matricen sendes til GPU'en
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(transform));
```

Nu da både model- og projektionsmatrixen er blevet sat for denne trekant, skal OpenGL blot have nogle punkter at tegne:

```
glDrawArrays(GL_TRIANGLES, 0, 3);
```

Disse punkter er blevet prædefineret, og kan genbruges til hver trekant. Genbrugen af geometri, som udelukkende differentiere i modelmatrix, kaldes for *instancing* indenfor 3D-grafik.

4.6 Delkonklusion

Resten af koden af udeladt, da det ikke omfatter lineær algebra, men kan alligevel blive fundet på [GitHub](#) eller på [side 19](#). Desuden kan en video af det endelige resultat blive set på [YouTube](#).

glm gør heldigvis den lineære algebra meget tilgængelig for programmører, men den underliggende teori er stadig den samme. Under de simple funktionskald såsom `glm::frustum`, `glm::translate` og `glm::rotate` konstrueres fortsat 4×4 matricer, som svarer direkte til de rumlige transformationer, der tidligere er blevet redegjort for. Programmet virker desuden som en opfølgelse på beviserne fra teorien, da det viser deres anvendelse i praksis.

Samtidigt er det tydeligt, at selvom biblioteker som *glm* abstraherer den matematiske kompleksitet fra programmøren, så er en grundlæggende forståelse af lineær algebra stadig nødvendig for at kunne anvende funktionerne korrekt. Uden kendskab til transformationernes betydning og rækkefølge ville det være vanskeligt at forudsige eller kontrollere det visuelle resultat.

Eksemplet er bevidst holdt simpelt og fokuserer udelukkende på den lineære algebra forbundet med *modeltransformation* og projektion. For at det blev et komplet spil, vil det også kræve virtuelt kamera. Dette er dog uden for opgavens omfang.

5 Diskussion

I det foregående er det blevet vist, hvordan lineær algebra danner det matematiske grundlag for rumlige transformationer i 3D-grafik, og hvordan disse principper kan implementeres i praksis ved hjælp af interfaces og moduler såsom OpenGL og *glm*. I dette afsnit diskuteres vil lineær algebras begrænsninger samt alternative tilgange til anvendelse af lineær algebra i grafiske applikationer og andre sammenhæng.

5.1 Styrker

Den fundamentale styrke ved lineær algebra ligger i dens mulighed for at beregnes parallelt. Derudover, som demonstreret i [Sætning 3.5.2](#), kan matricers egenskab til at sammensætte flere lineære transformation, reducerer mængden af beregninger betydeligt.

Dette gør lineær algebra fuldkommen nødvendigt når der er tale om realtids 3D-grafik. For at ramme gameres mindstekrav på 60 FPS⁴⁵, får grafikortet blot 16 ms pr. billede. Så når der er millioner af punkter, som skal transformeres hvert sekund, er det afgørende at operationerne kan distribueres til grafikortets tusinder af kerner. Den uafhængige natur af lineære transformationer betyder, at hvert punkt kan behandles parallelt, hvilket er præcis den type arbejde som GPU'er er optimeret til.

⁴⁵**Billeder pr. sekund**, en måleenhed for hvor mange gange et vindue/spil bliver tegnet i sekunder. Kan, ligesom *TFLOPS*, bruges til at måle hvor kraftfuld en GPU er.

Også selvom der ikke skal tegnes geometri, bliver lineær algebra brugt i mange andre sammenhæng for punkter skal transformeres, f.eks. ens browser, tegneprogrammer og fysikmotorer.

5.2 Begrænsninger

Selvom lineær algebra er kraftfuld og rasterisering har været brug i snart 30 år, har metoden også klare begrænsninger.

Den mest åbenlyse er, at lineære transformationer per definition, nødvendigvis er lineære. Dette betyder, at visse typer af deformationer, såsom bøjning, vridning eller ikke-uniform skalering langs kurver, ikke kan repræsenteres direkte i en 4×4 matrix.

En anden begrænsning ved rasterisering, som hele denne opgave bygger på, er dens fundamentale tilgang til belysning og skygger. Rasterisering behandler hver trekant isoleret og kan derfor ikke naturligt håndtere skygger, refleksioner, refraktioner eller indirekte belysning uden omfattende arbejde. Dette betyder i praksis, at naturlig korrekt belysning er stort set umuligt. Derfor bruger spil ofte mange tricks og illusioner til at implementere dette, såsom at 'bage' skygger ind i verdenen.

Den mest almindelige måde at lave realtids skygger med rasterisering hedder *shadowmapping*. I denne metode, vil verdenen blive tegnet fra hver lyskildes perspektiv. Alt geometri mellem lyskilden og det uendelige må kaste en skygge, hvilket bliver tegnet ind i en separat texture. Denne texture kan derefter projekteres fra lyskildens perspektiv, så den ses som skygge fra det virtuelle kamera.⁴⁶

5.3 Pathtracing

En mere radikal alternativ tilgang er at kombinere rasterisering med en helt anden tilgang. Pathtracing, også kendt som raytracing, som demonstreret i *Cyberpunk 2077*, ændrer fundamentalt på hvordan belysningen forgår i spillet. Trekanter bliver stadig rasterized, da det simpelthen ikke giver mening at bruge en anden metode til tegningen af geometri. Dog prøver metoden at simulere lysstrålernes vej gennem verden baglæns fra kameraet, resultatet af dette bruges til at belyse geometrien. Lineær algebra spiller stadig en central rolle her, da strålers retninger repræsenteres som vektorer og deres interaktion med overflader kræver transformation af normalvektorer. Forskellen ligger i at hver pixel beregnes baseret på hele scenens geometri, ikke kun de synlige overflader. Selvom teknologien er udviklet, så er det stadig kun de bedste (og dyreste) forbuger grafikkort som kan beregne dette alternativ.

I ikke-realtids sammenhæng bruges pathtracing ofte til at tegne billeder, på grund af dets naturlige belysning. Dette kunne f.eks. være i animerede film. Dette viser dog også hvor langsom denne metoder alene er. Firmaer såsom *Disney* og *Pixar* har flere hundrede af grafikkort og det kan stadig tage flere timer at renderes.⁴⁷

5.4 Perspektivering

Selvom denne opgave fokuserer på 3D-grafik, er det relevant at nævne lineær algebras bredere anvendelse. Netop fordi lineær algebra kan beregnes effektivt på GPU'er, har den stor anvendelse.

AI og *machine learning* er nok det mest fremtrædende eksempel. *Neurale netværk*, som skal repræsenterer en digital hjerne, består fundamentalt af lag af søjlematricer, der sammen med matrixmultiplikationer efterfulgt af ikke-lineære aktiveringsfunktioner, kan modelerer komplekse problemer. Det er de samme *tensor* kerner i moderne grafikkort, der bruges til 3D-grafik i spil, som også bruges til at træne AI-modeller.

⁴⁶(Gregory, 2018, Afsnit 11.3.3)

⁴⁷NVIDIA Blackwell Powers Real-Time AI for Entertainment Workflows - NVIDIA Blog

5.5 Fremtiden

Da rasterisering er så effektivt, kommer der til at gå lang tid før det erstattes. Men da standarden for videospil grafik kort ikke bliver nedsat, kommer hybridløsninger som i *Cyberpunk 2077*, til at blive mere udbredt.

Neural network-baserede teknikker vinder også frem i videospil grafik. Den mest kendte løsning til dette, NVIDIA's DLSS, anvender machine learning til at opskallere billeder fra lavere opløsninger, hvilket reducerer beregningsbelastningen markant.

Afslutningsvis kan det konkluderes, at selvom grafikteknologi udvikler sig hurtigt, forbliver lineær algebra det stabile fundament, uanset om rasterisering skulle forsvinde en dag. De matricer og transformationer som er blevet præsenteret i denne opgave vil være lige så relevante om ti eller tyve år, selvom konteksten måske ændrer sig. Nye interfaces vil komme og gå, hardware vil blive kraftigere, og nye rendering-teknikker vil blive opfundet, men de matematiske principper, der blev formaliseret i fra den teoretiske del vil forblive det samme.

6 Konklusion

I denne opgave er der blevet undersøgt, hvordan lineær algebra udgør det matematiske fundament for al moderne 3D-grafik i computerspil. Gennem vektorer og matricer muliggør lineær algebra rumlige transformationer og projektioner, som er afgørende for at kunne flytte, rotere, skalere og fremvise objekter fra en tredimensionel verden på en todimensionel skærm.

I den teoretiske gennemgang er der blevet redegjort for, hvordan lineære transformationer kan repræsenteres af matricer og sammensættes ved hjælp af matrixmultiplikation. Samt hvorfor homogene koordinater er nødvendige for at inkludere translation. Disse begreber er ikke blot abstrakte matematiske konstruktioner, som ligger over gymnasie-pensumet, men anvendes direkte i praksis i mange af de spil, der spilles i dag.

Analysen viser desuden, at lineær algebra er særligt velegnet til realtids 3D-grafik, på grund af dens evne til at blive beregnet parallelt på et grafikkort. Derudover blev alle de rumlige transformationer fra den teoretiske del vist i bevægelse ved hjælp af et eksempel program.

Der er blevet argumenteret både for og imod lineær algebras anvendelse i 3D-grafik. Rasterisering har klare begrænsninger, så snart at et højere grafik kvalitet bliver ønsket. Dette gælder især realistisk belysning som er en nødvendighed i moderne spil.

Samlet set kan det konkluderes, at selvom grafikteknologier og hardware udvikler sig hurtigt vil lineær algebra forblive uundværligt. Uanset om fremtidens computerspil benytter rasterisering, pathtracing eller andet, vil de grundlæggende matematiske principper fortsat spille en central rolle i udviklingen af interaktive 3D-verdener.

Kort om skrive processen: Opgaven er skrevet i markup sproget *Typst*, som er en spirituel efterfølger til $\text{\LaTeX}$. Til forskel fra *Word*, som har et WYSIWYG (hvad du ser, er hvad du får) layout, bliver man nødt til selv at sammensætte det **hele** med kommandoer. Selvom det kan virke mere besværligt, har det mange fordele. Layoutet kommer altid til være fuldkommen forudsigeligt og separeres fra selve indholdet af ens dokument, derved bliver skrive processen mere overskuelig og lettere at håndtere. Desuden kan man gøre brug af typst's mange moduler og templates⁴⁸ til at lave alt fra gantt-diagrammer, kvantekredsløb til figurer og modeller.

⁴⁸På nuværende tidspunkt er der 485 moduler og 495 templates i *Typst Universet*

Bibliografi

- borogk. (2021, maj 13). *Doom engine - Limited but still 3D* [Film]. YouTube. <https://www.youtube.com/watch?v=ZYGJQqhMN1U>
- De Vries, J. (2020). *Learn OpenGL: Learn modern OpenGL graphics programming in a step-by-step fashion* (Nos. ISBN: 978-90-90-33256-7). Kendall & Wells. <https://learnopengl.com/>
- eisfrosch. (2025, marts 7). *The Chaotic State of GPU Programming* [Film]. YouTube. <https://www.youtube.com/watch?v=9-DiGrnz8l8>
- Gregory, J. (2018). *Game engine architecture* (Nos. ISBN 978-1-138-3545-4(hbk). AK Peters/CRC Press.
- Jensen, M., Marthinus, K., & Hansen, B. (2019). *MAT A htx* (9788761693617). Systime. <https://mathtxa.systime.dk/?id=1>
- Jesper Funch Thomsen. (2020, december 18). *Lineær Algebra*. <https://data.math.au.dk/interactive/linalg/2020/>
- Koirikivi, R. (2015). *The architecture and evolution of computer game engines*. <https://urn.fi/URN:NBN:fi:oulu-201512112292>
- Lengyel, E. (2012). *Mathematics for 3D Game Programming and Computer Graphics* (ISBN-10: 1-4354-5886-9; 3. udg.). Course Technology.
- Lengyel, E. (2016). *Foundation of Game Engine Development: Mathematics* (ISBN-13: 979-8-9853582-0-9; Bd. 1). Terathon Software LLC.
- Lowood, H. (2014). *Game engines and game history* (Nos. ISSN 1916-985X; s. 179-198). Kinephanos. Revue d'études des médias et de culture populaire.

7 Bilag

Det følgende er programkoden til eksempel applikationen.

```
#include <glad/glad.h>
#include <GLFW/glfw3.h>
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>
#include <iostream>
#include <vector>

const char *vertexShaderSource =
"#version 460 core\n"
"layout (location = 0) in vec3 inPos;"
"layout (location = 1) in vec3 inColor;"
""
"uniform mat4 model;"
"uniform mat4 projection;"
""
"out vec3 outColor;"
""
"void main()\n"
"{\n"
"    gl_Position = projection * model * vec4(inPos, 1.0f);"
"    outColor = inColor;"
"}\0";

const char *fragmentShaderSource =
"#version 460 core\n"
"out vec4 FragColor;\n"
"in vec3 outColor;\n"
"void main()\n"
"{\n"
"    FragColor = vec4(outColor, 1.0f);\n"
"}\0";

void framebuffer_size_callback(GLFWwindow* window, int width, int height)
{
    glViewport(0, 0, width, height);
}

#define CHECK_SHADER(shader, type) \
{ \
    int success; \
    char infoLog[512]; \
    glGetShaderiv(shader, GL_COMPILE_STATUS, &success); \
    if (!success) \
    { \
        glGetShaderInfoLog(shader, 512, NULL, infoLog); \
        std::cout << "ERROR::SHADER::" << type << "::COMPILATION_FAILED\n" << infoLog \
<< std::endl; \
    } \
}

int SCREEN_WIDTH = 800;
int SCREEN_HEIGHT = 600;
```

```

int main() {
    glfwInit();
    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 4);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 6);
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);

    GLFWwindow* window = glfwCreateWindow(SCREEN_WIDTH, SCREEN_HEIGHT, "SOP - Linear
Algebra", NULL, NULL);
    if (window == NULL)
    {
        std::cout << "Failed to create GLFW window" << std::endl;
        glfwTerminate();
        return -1;
    }
    glfwMakeContextCurrent(window);

    if (!gladLoadGLLoader((GLADloadproc)glfwGetProcAddress))
    {
        std::cout << "Failed to initialize GLAD" << std::endl;
        return -1;
    }

    glViewport(0, 0, 800, 600);
    glfwSetFramebufferSizeCallback(window, framebuffer_size_callback);
    glClearColor(0.2f, 0.3f, 0.3f, 1.0f);

    float vertices[] = {
        -0.25f, -0.25f, 0.0f, 1.0f, 0.0f, 0.0f,
        0.25f, -0.25f, 0.0f, 0.0f, 1.0f, 0.0f,
        0.0f, 0.25f, 0.0f, 0.0f, 0.0f, 1.0f
    };

    unsigned int VAO;
    glGenVertexArrays(1, &VAO);
    glBindVertexArray(VAO);

    unsigned int VBO;
    glGenBuffers(1, &VBO);
    glBindBuffer(GL_ARRAY_BUFFER, VBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);

    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)0);
    glEnableVertexAttribArray(0);

    glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)(3 *
sizeof(float)));
    glEnableVertexAttribArray(1);

    unsigned int vertexShader;
    vertexShader = glCreateShader(GL_VERTEX_SHADER);
    glShaderSource(vertexShader, 1, &vertexShaderSource, NULL);
    glCompileShader(vertexShader);

    CHECK_SHADER(vertexShader, "VERTEX")

```



```

unsigned int fragmentShader;
fragmentShader = glCreateShader(GL_FRAGMENT_SHADER);
glShaderSource(fragmentShader, 1, &fragmentShaderSource, NULL);
glCompileShader(fragmentShader);

CHECK_SHADER(fragmentShader, "FRAGMENT")

unsigned int shaderProgram;
shaderProgram = glCreateProgram();
glAttachShader(shaderProgram, vertexShader);
glAttachShader(shaderProgram, fragmentShader);
glLinkProgram(shaderProgram);

CHECK_SHADER(shaderProgram, "PROGRAM")

glDeleteShader(vertexShader);
glDeleteShader(fragmentShader);

std::vector<glm::vec3> trianglePositions = {
    glm::vec3(-0.5f, 0.0f, 0.0f),
    glm::vec3(0.0f, 0.0f, 0.0f),
    glm::vec3(0.5f, 0.0f, 0.0f)
};

unsigned int modelLoc = glGetUniformLocation(shaderProgram, "model");
unsigned int projectionLoc = glGetUniformLocation(shaderProgram, "projection");

glm::mat4 perspective = glm::frustum(-1.0f, 1.0f, -1.0f, 1.0f, 1.0f, 10.0f);
glm::mat4 orthographic = glm::ortho(-1.0f, 1.0f, -1.0f, 1.0f, 1.0f, 10.0f);

while(!glfwWindowShouldClose(window))
{
    glClear(GL_COLOR_BUFFER_BIT);

    glUseProgram(shaderProgram);
    glBindVertexArray(VAO);

    float time = (float)glfwGetTime();
    for (size_t i = 0; i < trianglePositions.size(); ++i) {

        glm::mat4 projection;
        switch (i) {
            case 0:
                projection = perspective;
                break;
            case 1:
                projection = glm::identity<glm::mat4>();
                break;
            case 2:
                projection = orthographic;
                break;
        }
        glUniformMatrix4fv(projectionLoc, 1, GL_FALSE,
glm::value_ptr(projection));
    }
}

```

```

glm::mat4 transform = glm::identity<glm::mat4>();

if (i == 0 || i == 2) {
    // oscellate along z-axis
    float zdistance = sin(time) * 2.0f - 3.0f; // Move between -5 and -3
    transform = glm::translate(transform, trianglePositions[i] +
glm::vec3(0.0f, 0.0f, zdistance));
} else {
    transform = glm::translate(transform, trianglePositions[i]);
    float angle = (time + (float)i) * glm::radians(60.0f);
    transform = glm::rotate(transform, angle, glm::vec3(0.0f, 0.0f,
1.0f));

    float scaleAmount = sin(time + i) * 0.5f + 1.0f;
    transform = glm::scale(transform, glm::vec3(scaleAmount, scaleAmount,
scaleAmount));
}
glUniformMatrix4fv(modelLoc, 1, GL_FALSE,
glm::value_ptr(transform));

glDrawArrays(GL_TRIANGLES, 0, 3);
}

glfwSwapBuffers(window);
glfwPollEvents();
}

glfwTerminate();

return 0;
}

```